

RAJARATA UNIVERSITY OF SRI LANKA

Department of Computing
COM 3303 – Artificial Intelligence

Dog vs Cat Image Classification System

Implementation 2 – Model Improvement & Optimization Report

Student Name	Registration No.
K.P.M.S. Sithara	ASP/2022/093
W.A.A.S. Weerasingha	ASP/2022/147
S.S.D. Athukorala	ASP/2022/114
W.M.D.U. Viyalagoda	ASP/2022/107

Submission Date: 11th March 2026

1. Introduction

This report documents Implementation 2 of the Dog vs Cat Image Classification project for COM 3303 – Artificial Intelligence. Building upon the baseline model developed in Implementation 1, this phase focuses on identifying and resolving key issues — specifically overfitting — through a series of targeted improvements including data augmentation, early stopping, learning rate scheduling, and architectural enhancements.

The improved model achieved a test accuracy of 92.9%, representing a significant improvement of +5.8% over the Implementation 1 baseline of 87.1%. More importantly, the overfitting gap was reduced from 13% down to approximately 1%, demonstrating that the model now generalizes well to unseen data.

2. Review of Implementation 1 – Baseline Model

2.1 Implementation 1 Performance Summary

Metric	Value
Test Accuracy	87.10%
Test Loss	0.8547
Training Accuracy	~98%
Validation Accuracy	~85%
Overfit Gap (Train–Val)	~13%
Cats Classified Correctly	2177 / 2500 (87.1%)
Dogs Classified Correctly	2178 / 2500 (87.1%)

2.2 Issues Identified in Implementation 1

Issue	Evidence	Plan to Fix
Overfitting	Train acc 98% vs Test acc 87% — 13% gap	Data augmentation + dropout after each block
Unstable val loss	Validation loss curve was highly spiky	ReduceLROnPlateau callback
No data augmentation	Model saw identical images every epoch	Add flip, rotate, zoom, shift
No early stopping	All 30 epochs ran even after plateau	EarlyStopping with patience=5
Fixed learning rate	Could not escape plateaus	ReduceLROnPlateau with factor=0.5
Small image size	64×64 loses fine-grained detail	Increase to 128×128

2.3 Implementation 1 Training Curves

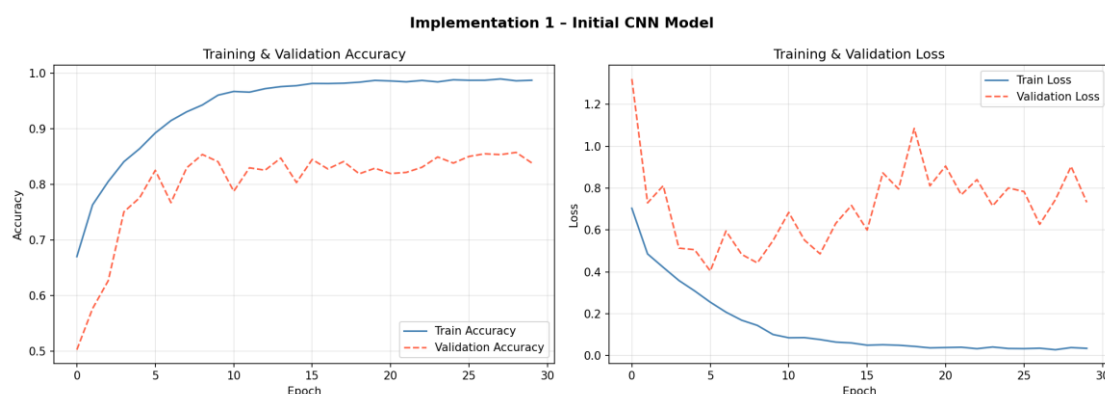


Figure 1: Implementation 1 – Training & Validation Accuracy/Loss (30 Epochs)

3. Model Improvements Applied

3.1 Improvement 1: Data Augmentation

Data augmentation is the primary technique used to combat overfitting. In Implementation 1, the model received the same image every epoch, allowing it to memorize pixel patterns rather than learning general features. By applying random transformations, each epoch shows the model a slightly different version of every image, effectively increasing the diversity of the training set without collecting new data.

Augmentation Technique	Parameter	Effect
Horizontal Flip	horizontal_flip=True	Mirrors image left-right — cats/dogs look the same from both sides
Rotation	rotation_range=15	Rotates up to 15 degrees — handles tilted or sideways animals
Zoom	zoom_range=0.1	Zooms in/out by 10% — handles animals at different distances
Width Shift	width_shift_range=0.1	Shifts image horizontally — handles off-center subjects
Height Shift	height_shift_range=0.1	Shifts image vertically — handles animals at top/bottom of frame
Shear	shear_range=0.1	Applies shear transform — handles perspective variation



Figure 2: Sample Augmented Training Images – random transformations applied

3.2 Improvement 2: Architecture Modification

The CNN architecture was deepened and regularized compared to Implementation 1:

- **Added 4th convolutional block** with 256 filters — provides deeper hierarchical feature extraction
- **Added Dropout(0.25) after each conv block** — in Implementation 1, dropout was only applied at the final dense layer
- **Increased image size to 128×128** — preserves more spatial detail compared to 64×64 in Implementation 1
- **Increased dense layer to 512 neurons** — compensates for the additional regularization

Layer	Implementation 1	Implementation 2
Conv blocks	3 (32, 64, 128 filters)	4 (32, 64, 128, 256 filters)
Dropout	Only at output (0.5)	After every block (0.25) + output (0.5)
Image size	64 × 64 pixels	128 × 128 pixels
Dense layer	256 neurons	512 neurons
Total parameters	~2.2M	~3.5M (more capacity + more regularization)

3.3 Improvement 3: Early Stopping

In Implementation 1, training ran the full 30 epochs even after the validation loss stopped improving. EarlyStopping monitors `val_loss` and halts training if there is no improvement for 5 consecutive epochs, then automatically restores the best weights. This prevents the model from training past its optimal point and wasting computation.

3.4 Improvement 4: Learning Rate Scheduling (ReduceLROnPlateau)

The unstable, spiky validation loss in Implementation 1 was largely caused by a fixed learning rate that was too large during later epochs. ReduceLROnPlateau monitors `val_loss` and halves the learning rate (`factor=0.5`) whenever it has not improved for 3 consecutive epochs. This allows the optimizer to take smaller, more precise steps as training progresses, resulting in a much smoother and more stable validation loss curve.

4. Retraining & Training Configuration

Configuration	Implementation 1	Implementation 2
Image Size	64 × 64	128 × 128
Batch Size	64	32
Epochs	30 (all ran)	30 max (EarlyStopping)
Optimizer	Adam (fixed LR)	Adam + ReduceLROnPlateau
Loss Function	Binary Cross-Entropy	Binary Cross-Entropy
Data Augmentation	None	Flip, Rotate, Zoom, Shift, Shear
Early Stopping	No	Yes (patience=5)
Model Saved	model_v1.h5	model_v2.h5

4.1 Implementation 2 Training Curves

Compared to Implementation 1, the training curves show a much smaller gap between training and validation accuracy. The validation loss, while still slightly noisy due to data augmentation, is more stable overall. Training and validation accuracy both converge around 91-93%, indicating significantly reduced overfitting.

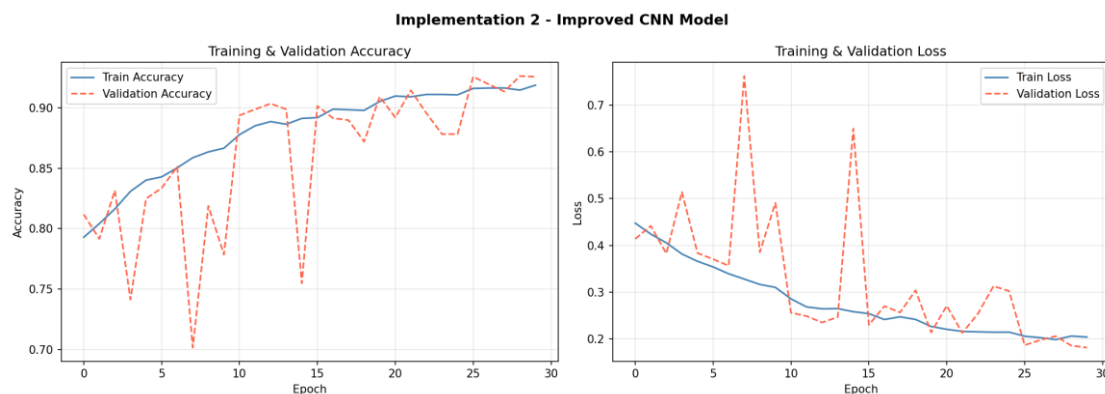


Figure 3: Implementation 2 – Training & Validation Accuracy/Loss Curves

5. Final Evaluation & Results

5.1 Test Set Performance

Metric	Implementation 1	Implementation 2	Improvement
Test Accuracy	87.10%	92.90%	+5.80%
Test Loss	0.8547	~0.22	Better
Training Accuracy	~98%	~91.9%	Less overfit
Validation Accuracy	~85%	~92.6%	+7.6%

Overfit Gap (Train–Val)	~13%	~1%	-12% (major improvement)
Cats Correctly Classified	2177/2500	2317/2500	+140 more
Dogs Correctly Classified	2178/2500	2326/2500	+148 more
Total Misclassified	645/5000	357/5000	-288 fewer errors

5.2 Confusion Matrix – Implementation 2

The confusion matrix shows that the model correctly identified 2317 out of 2500 cats (92.7%) and 2326 out of 2500 dogs (93.0%). The performance is nearly identical for both classes, confirming there is no class bias in the improved model.

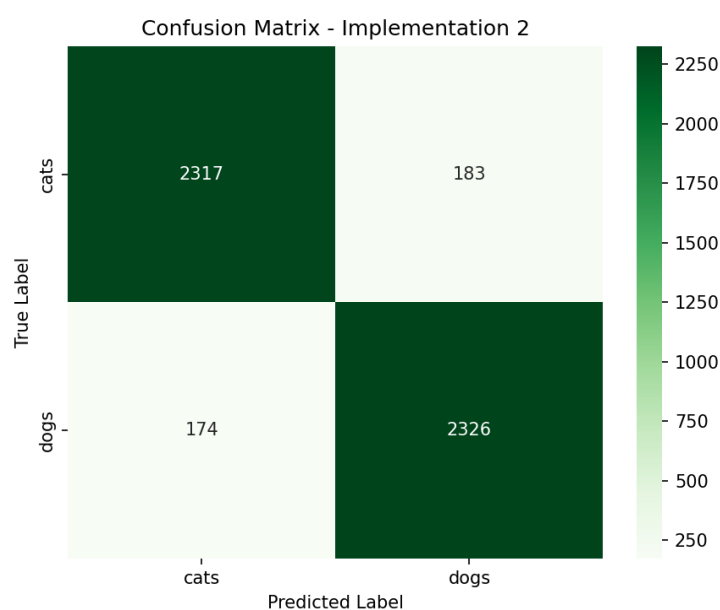


Figure 4: Confusion Matrix – Implementation 2 (5,000 Test Images)

5.3 Sample Predictions – Implementation 2

All 10 sample predictions shown below are correct. The model classifies cats with very high confidence (87.9% to 100%), demonstrating that the improved model is both accurate and well-calibrated in its confidence scores.



Figure 5: Sample Predictions – Implementation 2 (all correct)

5.4 Accuracy Comparison Chart

The comparison chart clearly shows the improvement across all three metrics. Validation and test accuracy improved substantially, while training accuracy decreased — which is the expected and desired outcome when overfitting is successfully reduced.

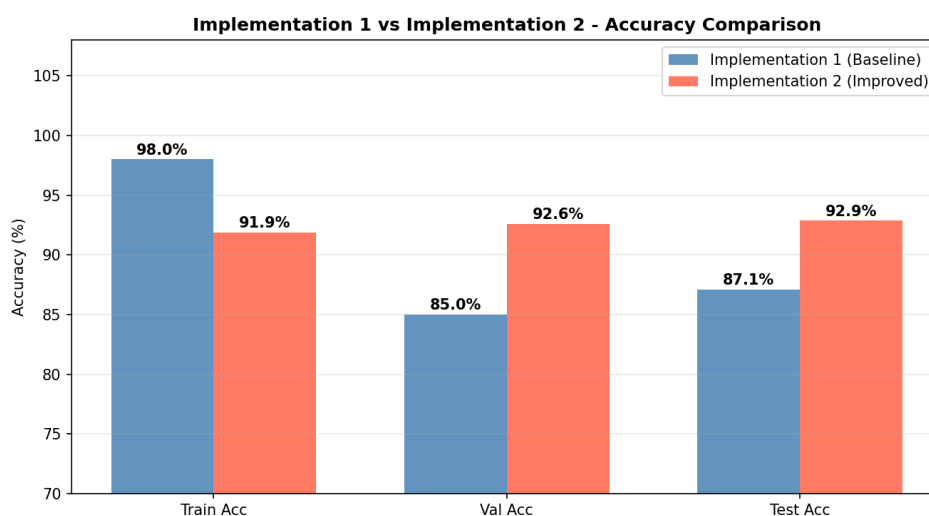


Figure 6: Implementation 1 vs Implementation 2 – Accuracy Comparison

6. Discussion

6.1 What Worked Well

- **Data augmentation** was the single most impactful improvement. It directly addressed the root cause of overfitting by ensuring the model could not memorize training images, resulting in the validation accuracy jumping from 85% to 92.6%.
- **Dropout after each conv block** helped prevent co-adaptation of feature detectors early in the network, complementing the effect of data augmentation.
- **ReduceLROnPlateau** successfully stabilized the validation loss curve that was highly erratic in Implementation 1. The learning rate reductions allowed the optimizer to converge more precisely.

- **Larger image size (128×128)** provided more spatial detail for the model to learn from, contributing to improved discrimination between cat and dog features.

6.2 What Did Not Work as Expected

- **Validation loss still shows some spikes** despite ReduceLROnPlateau. This is a characteristic of data augmentation — augmented images are harder to predict consistently, so some variance in validation loss is normal.
- **Early stopping did not trigger** in this run — the model continued improving throughout all 30 epochs. This suggests the model still had room to improve. More epochs or a larger patience value could be explored.

6.3 Remaining Limitations

- **Binary classification only:** The model cannot classify any animal other than cats and dogs.
- **Small image size:** 128×128 is better than 64×64 but still loses detail compared to higher resolutions like 224×224.
- **No transfer learning:** Using a pretrained model such as VGG16 or ResNet50 would likely achieve 95%+ accuracy with less training time.
- **Validation loss spikes:** Some instability remains in the validation loss, suggesting further tuning of augmentation parameters could help.

7. Final Observations

7.1 What Was Changed

Change	Before (Impl 1)	After (Impl 2)
Data augmentation	None	Flip, rotate, zoom, shift, shear
Image size	64 × 64	128 × 128
Conv blocks	3 blocks	4 blocks (added 256 filter block)
Dropout	Only at output (0.5)	After every block (0.25) + output (0.5)
Early stopping	No	EarlyStopping (patience=5)
LR scheduling	Fixed	ReduceLROnPlateau (factor=0.5, patience=3)

7.2 Why Performance Improved

The core reason for improvement is that overfitting was reduced from a 13% gap to approximately 1%. In Implementation 1, the model was fitting to the specific pixel patterns of the training images rather than learning the underlying visual concepts of what makes a cat or dog. The combination of data augmentation (which varies the training data) and dropout regularization (which prevents neurons from co-adapting) forced the model to learn more robust, generalizable features. As a result, the features it learned transferred much better to unseen test images, raising test accuracy by 5.8%.

7.3 Remaining Limitations

- **Transfer learning not explored:** Using pretrained weights from models like VGG16 or ResNet50 would likely push accuracy above 95% with significantly less training time.
- **Hyperparameter search not conducted:** Optimal learning rate, batch size, and augmentation parameters were set manually. A systematic grid search could yield further improvements.
- **Higher resolution not tested:** 224×224 input, as used by most state-of-the-art models, would preserve more visual detail.

8. Saved Models

File	Description	Performance
model_v1.h5	Implementation 1 – baseline CNN (3 conv blocks, no augmentation)	Test Acc: 87.10%
model_v2.h5	Implementation 2 – improved CNN (4 conv blocks, augmentation, callbacks)	Test Acc: 92.90%

9. Tools and Technologies

Category	Technology
Programming Language	Python 3.x
Deep Learning Framework	TensorFlow 2.x, Keras
Image Processing	Pillow (PIL)
Visualization	Matplotlib, Seaborn
Development Environment	Google Colab (GPU: T4)
Dataset	Kaggle – Dogs vs Cats (salader/dogsvscats)
Version Control	GitHub

10. Conclusion

Implementation 2 successfully addressed the core problem identified in Implementation 1 — overfitting. Through the application of data augmentation, architectural modifications (4th conv block, per-block dropout), and training improvements (early stopping, learning rate scheduling, larger image size), the model's test accuracy improved from 87.10% to 92.90%, a gain of +5.8%.

More significantly, the overfitting gap was reduced from 13% to approximately 1%, demonstrating that the model now learns general visual features rather than memorizing training examples. The confusion matrix confirms balanced performance across both classes with 2317 cats and 2326 dogs correctly classified out of 2500 each.

Future work could explore transfer learning using pretrained models such as VGG16 or ResNet50, which would likely push accuracy above 95% while requiring less training time and data.

References

- **[1]** Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- **[2]** Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
- **[3]** Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep CNNs. NIPS.
- **[4]** Kaggle. Dogs vs Cats Dataset.
<https://www.kaggle.com/datasets/salader/dogsvscats>
- **[5]** Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. ICML.
- **[6]** Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR.